



MINOR PROJECT REPORT ON

“ SORTING VISUALIZER ”

**SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by:

Rajnish kumar

Rollno.:12201056

SUBMITTED TO:

Er.Rakesh Singh

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNJABI UNIVERSITY

PATIALA – 147002

TABLE OF CONTENTS

1. CANDIDATE'S DECLARATION	3
2. ABSTRACT.....	4
3. INTRODUCTION	5
4. OBJECTIVE	6
5. FEATURES	7-8
6. Sorting Algorithms	9-10
7. PROJECT FLOWCHART	11
8. TOOLS AND TECHNOLOGIES USED	12-14
9. RESULTS	15-16
10. CONCLUSION.....	17
11. REFERENCES	18

CANDIDATE’S DECLARATION

I Rajnish kumar, hereby declare that the project report entitled “Sorting Visualizer Web Application” submitted by me to the Department of Computer Science and Engineering, Punjabi University, Patiala, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out by me under the guidance of [Er.Rakesh Singh].

Signature of the Student

**Rajnish kumar
(12201056)**

The Minor Project Viva-Voce Examination of

has been held on _____ and accepted

Signature of the Examiner

Abstract

The Sorting Visualizer is a web-based application built using HTML, CSS, and JavaScript to demonstrate the working of various sorting algorithms interactively. This tool provides users with a visual representation of sorting techniques such as Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and more. The platform allows users to input custom arrays or generate random datasets, adjust sorting speed, and observe the step-by-step execution of algorithms.

With a user-friendly interface and real-time animations, this application enhances learning by making complex sorting algorithms easy to understand. Users can compare different sorting methods based on their time complexity and efficiency through side-by-side visualization. The application is designed to be responsive, ensuring compatibility across multiple devices.

By providing an interactive experience, the Sorting Visualizer serves as an educational tool for students and enthusiasts who want to grasp the fundamentals of sorting algorithms in an engaging and effective manner.

INTRODUCTION

Computer Graphics :-

Graphics provides one of the most natural means of communicating with a computer, since our highly developed 2D and 3D pattern recognition abilities allow us to perceive and process pictorial data rapidly and efficiently. Interactive computer graphics is the most important means of producing pictures since the invention of photography and television. It has the added advantage that, with the computer, we can make pictures not only of concrete real-world objects but also of abstract, synthetic objects, such as mathematical surfaces and of data that have no inherent geometry, such as survey results.

Overview of Project :-

This project discusses a study performed on animating sorting algorithms as a learning aid for classroom instruction. The web-application animation tool was created to visualize six common sorting algorithms: Selection Sort, Bubble Sort, Insertion Sort, Heap Sort, Quick Sort, and Merge Sort. The animation tool represents data as a bar-graph and dot plot. After selecting a data ordering and algorithm, the user can run an automated animation or step through it at their own pace.

OBJECTIVE

The primary objective of this project is to provide a clear and interactive visualization of various sorting algorithms, helping learners understand how these algorithms work internally. The key goals include:

- To develop a web-based sorting visualizer using HTML, CSS, and JavaScript.
- To animate the step-by-step process of sorting data using six algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.
- To make learning sorting algorithms more engaging and accessible for students and beginners in computer science.
- To allow users to control the size of the array and the speed of the sorting animation.
- To provide a clean and user-friendly interface that aids in better understanding of time and space complexities of different sorting techniques.

This tool aims to bridge the gap between theoretical algorithm learning and practical understanding through visual interaction.

FEATURES

The Sorting Visualizer project comes with several interactive and educational features designed to enhance the learning experience of sorting algorithms. The main features of the project include:

- **Array Generation:** Users can generate random arrays of adjustable sizes with just one click. The visual bars representing array elements are generated dynamically.
- **Adjustable Speed Control:** The animation speed of the sorting process can be modified by the user using a slider, allowing better observation of each step.
- **Algorithm Selection:** The visualizer includes six widely used sorting algorithms:
 1. Bubble Sort
 2. Selection Sort
 3. Insertion Sort
 4. Merge Sort
 5. Quick Sort
 6. Heap Sort
- **Visual Representation:** Array elements are shown as vertical bars, and their height represents the value. During the sorting process, color changes are used to highlight comparisons, swaps, and sorted elements.
- **Responsive Layout:** The interface is fully responsive and works well across various screen sizes and devices.

- **User-friendly Interface:** All buttons, sliders, and interactions are designed for clarity and ease of use, even for beginners.
- **Educational Animation:** The visualization helps users understand the inner workings of each algorithm by showing how data moves step-by-step.

These features make the project a powerful educational tool for students learning Data Structures and Algorithms.

Sorting Algorithms

Here are explanations for each of the sorting algorithms implemented in the Sorting Visualizer project:

a. Bubble Sort

Bubble Sort is a simple sorting algorithm that works by repeatedly swapping adjacent elements that are in the wrong order. The algorithm iterates through the array multiple times until it is sorted. In each iteration, it compares adjacent elements and swaps them if necessary. After each pass, the largest unsorted element "bubbles up" to its correct position at the end of the array.

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the number of elements.

b. Selection Sort

Selection Sort works by repeatedly finding the minimum element from the unsorted portion and placing it at the beginning. It divides the array into two parts: sorted and unsorted. During each iteration, the minimum element is selected from the unsorted part and swapped with the leftmost unsorted element.

Time Complexity: $O(n^2)$ in the worst-case scenario.

c. Insertion Sort

Insertion Sort divides the array into a sorted and an unsorted part. It picks elements from the unsorted part one by one and places them at the correct position in the sorted part. The sorted part starts with the first element and grows with each iteration.

Time Complexity: $O(n^2)$ in the worst-case scenario.

d. Heap Sort

Heap Sort uses a binary heap (generally a max-heap) to sort elements. It starts by building a max-heap from the input data. Then, the largest element (root of the heap) is swapped with the last element, and the heap size is reduced. This process is

repeated until the heap is empty and the array is sorted.

Time Complexity: $O(n \log n)$ in the worst-case scenario.

e. Merge Sort

Merge Sort is a divide-and-conquer algorithm. It divides the array into two halves, recursively sorts each half, and then merges the sorted halves. The merging step combines the two sorted arrays into a single sorted array by comparing the elements one by one.

Time Complexity: $O(n \log n)$ in the worst-case scenario.

f. Quick Sort

Quick Sort is also a divide-and-conquer algorithm that selects a pivot element and partitions the array around the pivot. Elements smaller than the pivot are moved to its left, and larger ones to its right. This process is repeated recursively for each partition. The efficiency depends on the choice of pivot.

Time Complexity: Average case is $O(n \log n)$, but the worst-case is $O(n^2)$ when pivot selection is poor.

PROJECT FLOWCHART

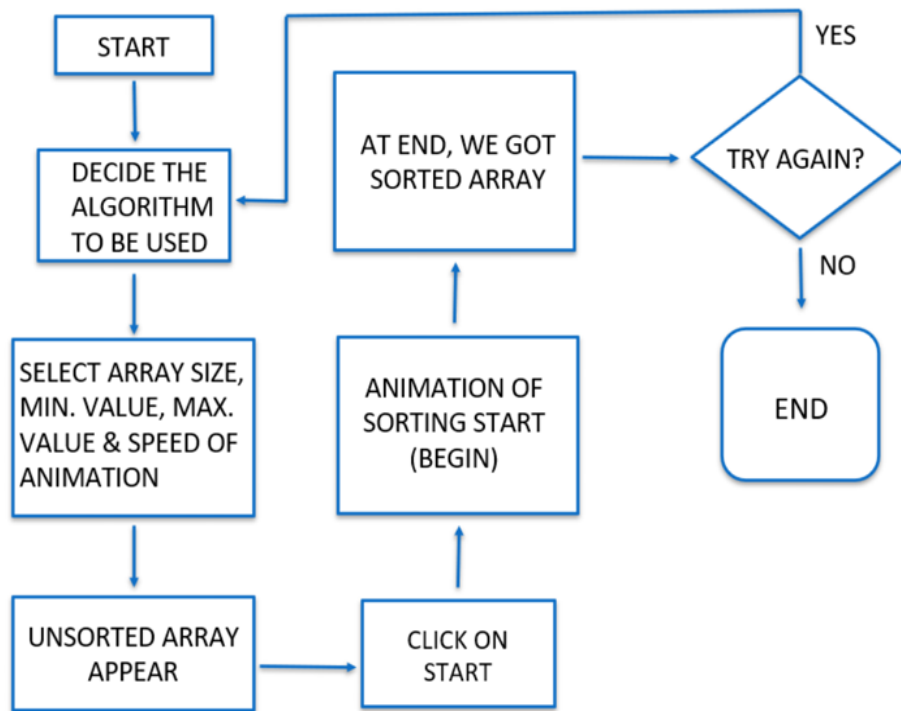


Figure- Project Flow chart

TOOLS AND TECHNOLOGIES USED

To develop the Sorting Visualizer web application, a combination of modern web development technologies and tools was used to ensure efficiency, interactivity, and responsiveness. The following are the core tools and technologies utilized in the project:

🔗 HTML (HyperText Markup Language):

HTML is the foundation of the project, providing the structure and elements of the web page. It helps in defining the layout and content, including buttons, div containers, and text, ensuring that the content is properly displayed in the browser.



Fig 1.HTML 5

🔗 CSS (Cascading Style Sheets):

CSS is used for styling the HTML elements. It controls the appearance of the page, including colors, layouts, fonts, and spacing. For this project, CSS helps in creating the visual layout, styling the bars, buttons, and overall presentation.



Fig 2.CSS

? JavaScript:

JavaScript is the core programming language used for implementing the sorting algorithms and animation logic. It controls the interactive elements of the page, such as the sorting animations, user inputs, and button functionality. JavaScript is also used to dynamically generate the bars and control the speed of the animation.



Fig 3.JAVA SCRIPT

? Web Browser (Google Chrome):

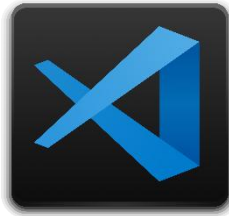
The project is designed to be run on a modern web browser, with Google Chrome being one of the primary platforms used for testing. The web browser interprets the HTML, CSS, and JavaScript to render the sorting visualizer and execute the logic.



Fig 4.Chrome

? Visual Studio Code (VS Code):

Visual Studio Code is used as the integrated development environment (IDE) for coding the project. It provides a user-friendly interface, code syntax highlighting, debugging tools, and version control to facilitate smooth development.



Results

The Sorting Visualizer project successfully demonstrates the working of six fundamental sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.

Key Highlights:

- Dynamic Array Generation
- Speed Adjustment Feature
- Real-Time Visual Sorting
- Responsive Design

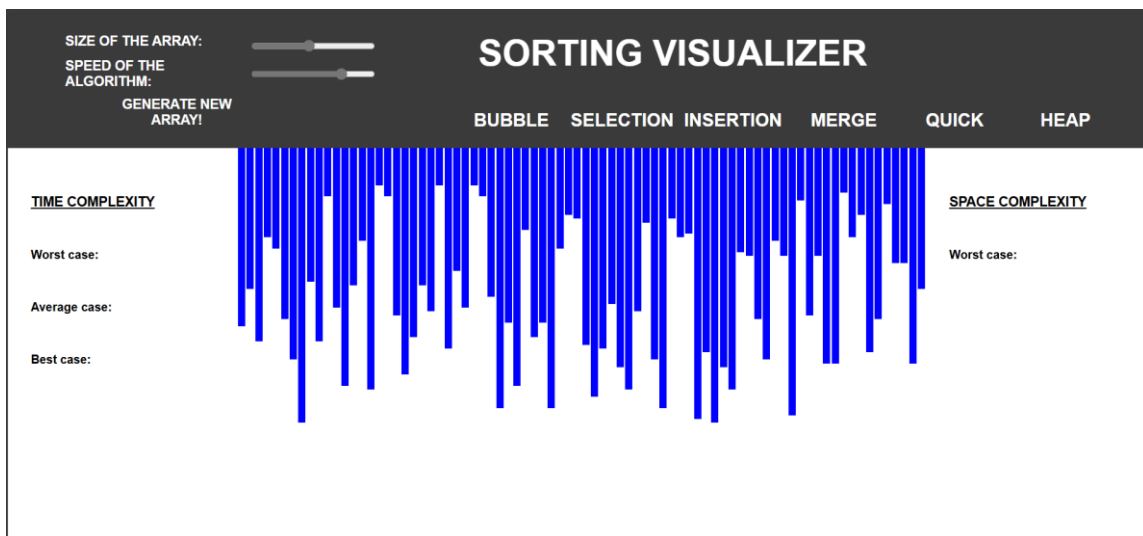


Figure 1:- FrontPage Before Sorting

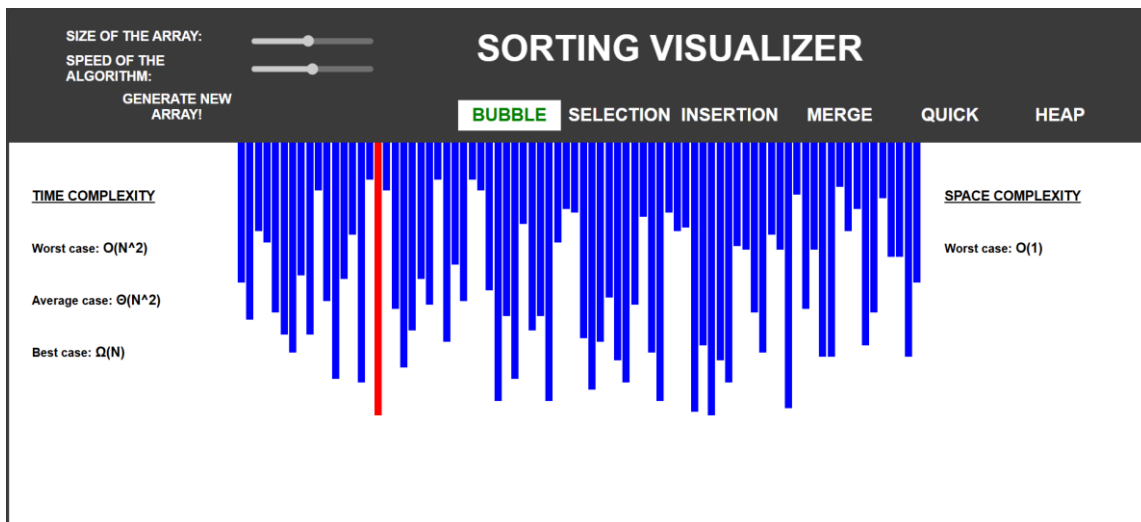


Figure 2: Bubble Sort in Action

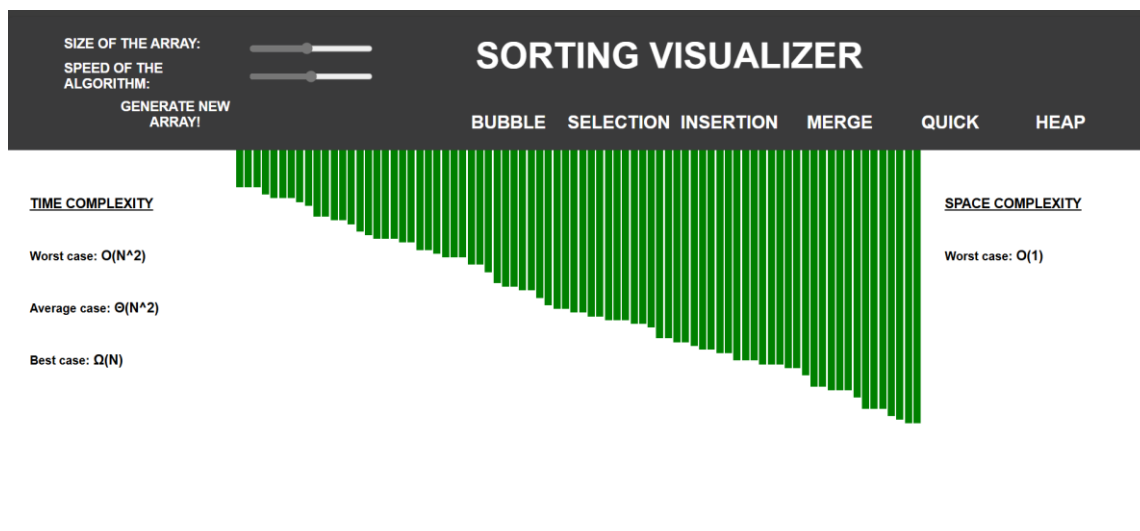


Figure 3: Sorted Array Elements

Conclusion

The Sorting Visualizer project successfully fulfills its objective of providing a clear, interactive, and educational tool to understand how different sorting algorithms work. Through real-time animations and a user-friendly interface, users can observe how each algorithm behaves, compares elements, and performs sorting operations. The implementation of six fundamental sorting algorithms — Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort — offers a comprehensive learning experience.

This project not only improved my understanding of JavaScript, DOM manipulation, and algorithmic thinking but also enhanced my skills in frontend development and debugging. Overall, the Sorting Visualizer stands as a practical and engaging way to bridge the gap between theory and visualization in computer science.

References

The following resources were referred to during the development of the Sorting Algorithm Visualizer project:

- [MDN Web Docs – JavaScript Documentation](#)
- [W3Schools – HTML, CSS, and JavaScript Tutorials](#)
- **GeeksforGeeks – Sorting Algorithm Explanations**
- [FreeCodeCamp – JavaScript Sorting Videos and Articles](#)
- **YouTube Tutorials on Sorting Algorithms (Various Creators)**
- **Stack Overflow Discussions for Debugging Assistance**
- **Personal Notes and Online Articles on Frontend Development**

These references helped in understanding the logic, improving design, and troubleshooting during the project.